TECHNISCHE UNIVERSITÄT
ILMENAU

Institut für Technische Informatik und Ingenieurinformatik
Fakultät für Informatik und Automatisierung
Fachgebiet System- und Software-Engineering

Bachelorarbeit

Modellgetriebene Microservice-Entwicklung

vorgelegt von

Patrik Löffler
Matrikelnummer 56207

Betreuer:

Prof. Armin Zimmermann
Dr. Ralph Maschotta, Tino Jungebloud

Ilmenau, den 19. Oktober 2023

Inhaltsverzeichnis

1 Einleitung **1**

1.1 Einführung in das Thema 1

1.2 Zielstellung 1

2 Grundlagen **3**

2.1 Domain-Driven Design 3

2.1.1 Definition 3

2.1.2 Bausteine des Model-Driven Design 4

2.1.3 Strategic Design 8

2.2 Microservices 11

2.2.1 Definition 11

2.2.2 Architektur von Microservice-Systemen 12

2.2.3 Kommunikation 14

2.2.4 Architektur eines Microservices 15

2.2.5 Betrieb und Continious Delivery 16

2.3 Cloud-Infrastruktur 17

2.3.1 Definition 17

2.4 Modellgetriebene Entwicklung von Microservices 18

2.4.1 AjiL - an Eclipse tool for MSA modeling with a code generator 18

2.4.2 OCCIware - a unified framework for managing diverse cloud resources and services 18

2.4.3 ARGON - automated cloud infrastructure provisioning using a domain-specific language 19

2.4.4 model-driven approach for adaptive microservice-based systems using VxBPMN4MS 19

2.4.5 Context Mapper, a tool for DDD and microservice design. 20

2.4.6 DOML simplifies infrastructure deployments by unifying multiple IaC languages 20

1 Einleitung

1.1 Einführung in das Thema

1.2 Zielstellung

Ziel der Arbeit ist es, Anwendungen, die das Microservice-Paradigma verwenden, mit Werkzeugen und Methoden der modellgetriebenen Softwareentwicklung zu entwickeln.

Dazu wird ein Metamodell konzipiert, welches die grundlegenden Eigenschaften des Paradigmas abstrahiert. Ergänzt wird dies durch die Berücksichtigung von Konzepten des Domain-Driven Design zur Modellierung der Geschäftslogik.

Die mithilfe dieses Metamodells generierten Anwendungen sollen in der Lage sein, bereits existierende, sogenannte Microservice Chassis – wiederverwendbare Grundbausteine eines Microservices – zu verwenden. Daraufhin wird erforscht, welche Möglichkeiten es gibt, diese generierten Anwendungen automatisiert, in Abhängigkeit von Versionsaktualisierungen der Chassis, zu migrieren.

Außerdem wird untersucht, inwiefern die der Anwendung zugrunde liegenden IT-Infrastruktur in solch einem Metamodell berücksichtigt werden kann und welche Möglichkeiten sich daraus ergeben.

2 Grundlagen

2.1 Domain-Driven Design

2.1.1 Definition

Modelle sind bewusst simplifizierte und strukturierte Darstellungen von Wissen. Die Methodik des Domain-Driven Design (DDD) stellt drei Regeln auf, welche bei der Wahl eines Modells zu berücksichtigen sind: Modell und Implementierung beeinflussen sich gegenseitig. Das Modell ist die Grundlage der im Entwicklungsprozess verwendeten Terminologie. Das Modell beinhaltet Wissen in einer nutzbaren, aufgearbeiteten Form. [Eva07, S. 3-4]

Fachlichkeiten aus einer Domäne werden mit einer Unbiquitous Language, einer dem Modelbasieten Sprache welche Entwickler und Domänen-Experten eine gemeinsame Terminologie vorgibt, ausgedrückt. [Eva07, S. 25-27]

Im Domain-Driven Design (DDD) bezeichnet und gewährleistet das Model-Driven Design(MDD) die möglichst enge Abstimmung zwischen Software-Design und konzeptionellem Domänenmodell. Um eine hohe Übereinstimmung zwischen Modell und Design zu erreichen, ist es fast unerlässlich, innerhalb eines von Software-Werkzeugen unterstützten Modellierungsparadigmas zu arbeiten. Ein ideales Werkzeug sind objektorientierte Programmiersprachen, da Modellkonzepte sich recht gut in objektorientierte Konzepte übersetzen lassen. [Eva07, S. 49-51]

2.1.2 Bausteine des Model-Driven Design

Folgende abstrakten Bausteine existieren im MDD:

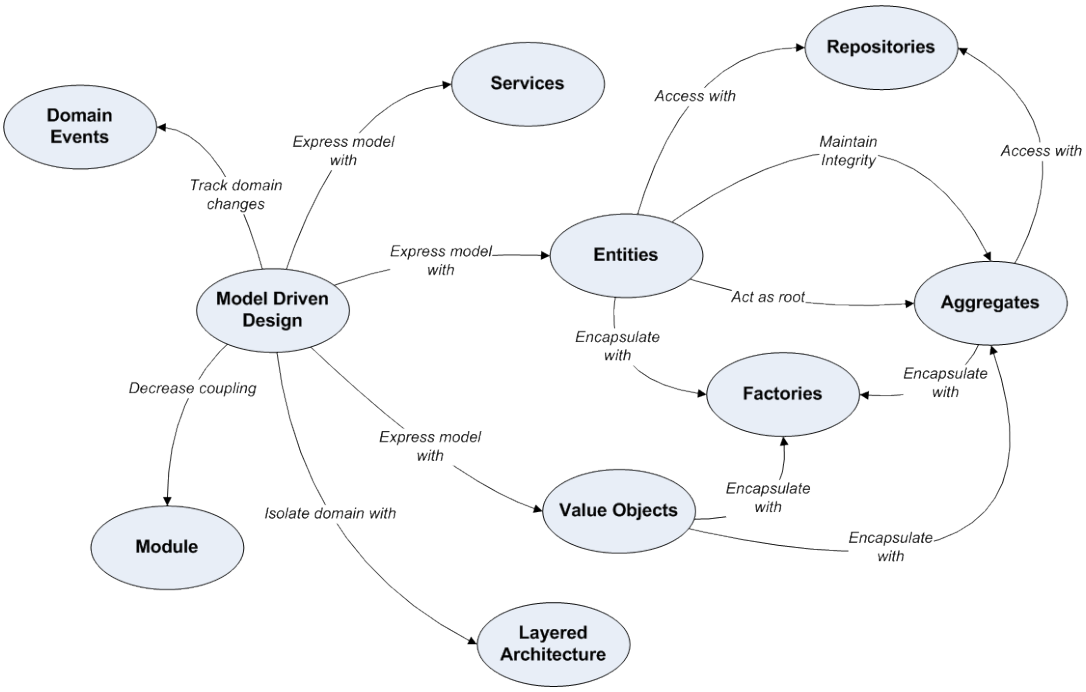

Abbildung 2.1: Bausteine des Model-Driven Design

[Eva07, S. 65]

Schichtenarchitektur

Um die Komplexität von Software zu beherrschen, werden unter anderem Schichtenarchitekturen verwendet. In dieser hängt jedes Element einer Schicht nur von anderen Elementen in der gleichen oder den darunterliegenden Schichten ab. Im Kontext des DDD ist hier die Trennung der Geschäftslogik in eine eigene Domain Layer entscheidend. Dies erzeugt eine starke Kohäsion zum Modell im Sinne des MDD. Mit Entwurfsmustern wie dem Model-View Controller kann auf diese Domain Layer wieder zugegriffen werden. [Eva07, S. 69 - 73]

Associations

Die Umsetzung von Assoziationen kann zwischen Modell und Implementierung stark variieren. Um die Assoziationen eines Modells besser handhabbar zu machen, gibt es verschiedene Ansätze: Das Festlegen einer Durchlaufrichtung verineinfacht beispielsweiße das Design und Reduziert Abhäbgigkeiten. Auch das Hinzufügen eines Qualifikators der effektiv die Multiplizität reduziert vereinfacht Modell und Implementierung. Das Entfernen von nicht essenziellen Assoziationen ist schließlich ebenso möglich. [Eva07, S. 81-85]

Entities

Ein Objekt, das hauptsächlich durch seine Identität definiert wird, wird als Entität bezeichnet. Dessen Attribute sind dabei nebensächlich. Eine Entität stellt eine abstrakte Kontinuität eines Objektes dar, die sich durch einen Lebenszyklus zieht. Klassendefinitionen, Verantwortlichkeiten, Attribute und Assoziationen sollten die Identität der Entität repräsentieren.

Bei einer Modellierung ist die grundlegendsten Verantwortlichkeit von Entitäten, Kontinuität herzustellen. Sie sollten sparsam verwendet werden. Nur das Verhalten das unerlässlich ist sollte ihnen angefügt werden. Darüber hinaus sollten zusätzliche Verhalten und Attribute zu anderen damit verbundenen Objekten verschoben werden. Die Identität soll mit einem einzigartigen Identifikationswert, welcher beispielsweiße automatisch generiert werden kann, sichergestellt werden. [Eva07, S. 89-96]

Value Objects

Ein Objekt, das einen beschreibenden Aspekt des Domänenbereichs darstellt und keine konzeptionelle Identität besitzt, wird als Wertobjekt (Value Object) bezeichnet. Wertobjekte können eine Zusammenstellung von anderen Objekten sein. Sie können Entitäten referenzieren. Wertobjekte werden häufig als Parameter in Nachrichten zwischen Objekten übergeben. Sie werden als Attribute von Entitäten (oder anderen Werten) verwendet. Die Felder eines Werteobjekts sollen ein konzeptionelles Ganzes formen.

Beim Design sollte das Verhalten beim kopieren, teilen und der Veränderlichkeit von Value Objekten berücksichtigt werden. Damit Value Objects sicher geteilt werden können müssen sie unveränderlich sein. Alternativ müssen Value Objekte als Kopien übergeben werden. Weiterhin sind bidirektionale Assoziationen zwischen Value Objekten aufgrund der fehlenden Identität bedeutungslos und sollten entfernt werden. [Eva07, S. 97-103]

Services

Einige Konzepte aus dem Domänenbereich sind nicht natürlich als Objekte zu modellieren. Daher wird ein neues Konzept eingeführt: Ein Service ist eine als Schnittstelle angebotene Operation, die im Modell eigenständig steht, ohne einen Zustand zu kapseln. Er wird rein nach dem definiert, was er für einen Client tun kann. Meist wird er nach einer Aktivität benannt und nicht nach einer Entität. Sie sollten bedacht eingesetzt werden und es sollte nicht zugelassen werden, dass sie den Entitäten und Wertobjekten all ihr Verhalten entziehen. Ein guter Service hat drei Eigenschaften: Die Operation bezieht sich auf ein Domänenkonzept, das kein natürlicher Teil einer Entität oder eines Wertobjekts ist. Die Schnittstelle ist in Bezug auf andere Elemente des Domänenmodells definiert. Die Operation ist zustandslos. Da Services auch außerhalb der Domain Layer auftreten können ist eine präzise fachliche Betrachtung nötig. [Eva07, S. 104 - 108]

Modules/Packages

Modules strukturieren Teile des Modells, sodass in ihnen eine möglichst hohe Kohäsion herrschst und zwischen ihnen eine möglichst lose Kopplung. Daher bündeln sie konzeptionell zusammengehörige Elemente. [Eva07, S. 109 - 116]

Aggregates

Ein Aggregat ist ein Cluster von assoziierten Objekten, die wir zu dem Zweck von Datenänderungen als eine Einheit behandeln. Jedes Aggregat hat eine Wurzel und eine Grenze. Die Grenze definiert, was sich innerhalb des Aggregats befindet. Die Wurzel ist eine einzelne, spezifische Entität innerhalb des Aggregats. Die Wurzel ist das einzige Mitglied des Aggregats, auf das Außenobjekte Referenzen halten dürfen, obwohl Objekte innerhalb der Grenze Referenzen untereinander halten dürfen. Entitäten, die nicht die Wurzel sind, haben eine lokale Identität, aber diese Identität muss nur innerhalb des Aggregats unterscheidbar sein, da kein Außenobjekt sie außerhalb des Kontexts der Wurzel-Entität sehen kann. [Eva07, S. 123 - 135]

Factories

Die Erstellung komplexer Objekte ist eine Verantwortung der Domänenschicht, dennoch gehört diese Aufgabe nicht zu den Objekten, die das Modell ausdrücken. Ein Programmelement, dessen Verantwortung die Erstellung anderer Objekte ist, wird als Fabrik (Factory) bezeichnet. Eine Fabrik kapselt das Wissen ein, das zur Erstellung eines komplexen Objekts oder Aggregats benötigt wird. Zwei grundlegende Anforderungen sind:

Jede Erstellungsmethode ist atomar und erzwingt alle Invarianten des erstellten Objekts oder Aggregats. Eine Fabrik sollte nur in der Lage sein, ein Objekt in einem konsistenten Zustand zu produzieren.

Die Fabrik sollte abstrakt auf den gewünschten Typ ausgerichtet sein, anstatt auf die konkrete(n) erstellte(n) Klasse(n).

Obwohl eine Fabrik dafür verantwortlich ist sicherzustellen, dass alle Invarianten erfüllt sind, kann sie die Überprüfung von Invarianten delegieren, was oft am besten ist. Eine Fabrik kann für die Erstellung oder Rekonstitution verwendet werden. Die zweite hat zwei wesentliche Unterschiede:

Eine Entitäten-Fabrik, die zur Rekonstitution verwendet wird, weist keine neue Tracking-ID zu.

Eine Fabrik, die ein Objekt rekonstituiert, wird Verstöße gegen eine Invariante anders behandeln, da ein reines Ignorieren möglicherweise nicht ausreicht. [Eva07, S. 136 - 146]

Repositories

Ein Repository stellt alle Objekte eines bestimmten Typs als ein konzeptuelles Set dar. Es verhält sich wie eine Sammlung, jedoch mit einer ausgefeilteren Abfragefähigkeit. Objekte des entsprechenden Typs werden hinzugefügt und entfernt, und die Maschinerie hinter dem Repository fügt sie in die Datenbank ein oder löscht sie daraus. Clients fordern Objekte aus Repositories an, indem sie Abfragemethoden verwenden, die Objekte basierend auf vom Client angegebenen Kriterien auswählen, wie zum Beispiel Werte bestimmter Attribute. Diese Abfragen können in ihrer Komplexität stark variieren. [Eva07, S. 147 - 161]

2.1.3 Strategic Design

Die strategischen Designprinzipien müssen Designentscheidungen leiten, um die Abhängigkeit der Teile zu verringern und die Klarheit zu verbessern, ohne dabei die kritische Interoperabilität und Synergie zu verlieren. [Eva07, S. 334]

Strateic Design bezeichnet wie verschiedene Domänenmodelle miteinander interagieren können. [Wol16, S. 45]

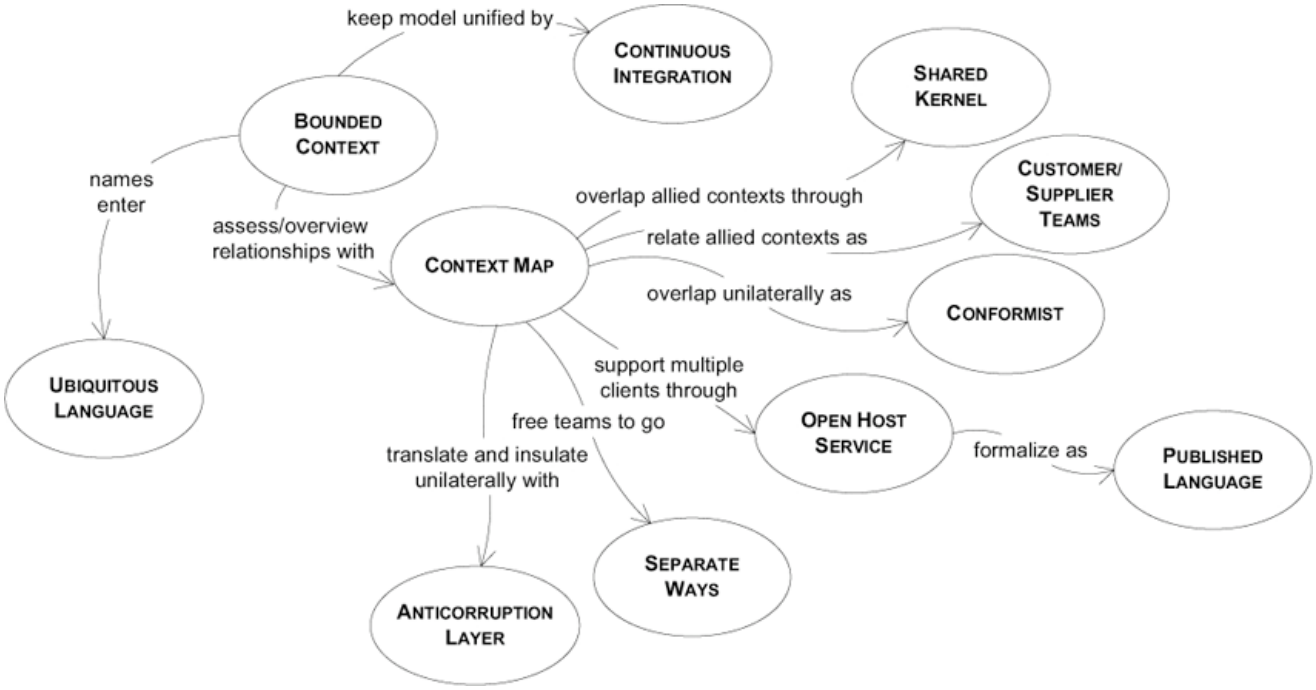

Abbildung 2.2: Supple Design

Bounded Context

Ein Modell findet in einem Kontext Anwendung. Der Kontext kann ein bestimmter Teil des Codes oder die Arbeit eines bestimmten Teams sein. Der Modellkontext sind alle Bedingungen, die erfüllt sein müssen, damit man sagen kann, dass die Begriffe in einem Modell eine spezifische Bedeutung haben.

Ein abgegrenzter Kontext begrenzt die Anwendbarkeit eines bestimmten Modells, sodass Teammitglieder ein klares und gemeinsames Verständnis davon haben, was konsistent sein muss und wie es sich zu anderen Kontexten verhält. Innerhalb dieses Kontexts sollte daran gearbeitet werden, das Modell logisch einheitlich zu halten, aber man sollte sich keine Gedanken über die Anwendbarkeit außerhalb dieser Grenzen machen. [Eva07, S. 335 - 340] [Wol16, S. 45]

Context Map

Eine Kontextkarte befindet sich in der Überschneidung zwischen Projektmanagement und Softwareentwurf. Es handelt sich um eine Karte des bestehenden Geländes von Modellen mit Schwerpunkt auf den Berührungspunkten, die eine explizite Übersetzung für jegliche Kommunikation skizziert und jegliches Teilen hervorhebt. [Eva07, S. 344 - 353] [Wol16, S. 45]

Shared Kernel

Ein Teilbereich des Domänenmodells kann dafür vorgesehen werden, unter Teams geteilt zu werden. Dies umfasst die Codebasis und die Datenbank, die möglicherweise mit dem Modell verbunden sind. Dieser geteilte Kern (Shared Kernel) sollte sorgfältig behandelt werden, da daran mehr Entwickler beteiligt sind. Der Shared Kernel ist oft die Kern-Domäne (Core Domain), eine Reihe von generischen Teil-Domänen (Subdomains) oder beides. Das Ziel ist es, Duplikationen zu reduzieren. [Eva07, S. 354 - 355] [Wol16, S. 45]

Customer/Supplier

Zwischen Parteien, die zwei unterschiedliche begrenzte Kontexte aufrechterhalten, kann eine Beziehung in Form einer nachgelagerten (downstream) und vorgelagerten (upstream) Komponente entstehen. Daher führt das Domain-Driven Design eine klare Kunden-/Lieferantenbeziehung zwischen solchen Kontexten ein. Daher erfüllt ein nachgelagertes Fragment eine kundenähnliche Rolle mit eigenen Bedürfnissen, die das vorgelagerte Fragment berücksichtigen muss. Automatisiertes Testen und Überwachen müssen implementiert werden, um zu verhindern, dass der nachgelagerte Bereich beschädigt wird. [Eva07, S. 356 - 360] [Wol16, S. 45-47]

Conformist

Wenn die Bedürfnisse eines solchen nachgelagerten, abhängigen Subsystems durch den vorgelagerten Teil erfüllt werden können, ist eine mögliche Lösung, die Rolle des Konformisten der nachgelagerten Komponente zuzuweisen. Dadurch wird die Komplexität der Übersetzung zwischen den abgegrenzten Kontexten beseitigt, indem man sich an das Modell des vorgelagerten Fragments hält. [Eva07, S. 361 - 363] [Wol16, S. 47]

Anticorruption Layer

Wenn Systeme, die auf unterschiedlichen Modellen basieren, kombiniert werden, kann die Notwendigkeit, dass das neue System sich an die Semantik des anderen Systems anpasst, zu einer Korruption des eigenen Modells des neuen Systems führen. Eine Antikorruptionsschicht kann erstellt werden, um eine isolierende Schicht zu schaffen, die den Clients Funktionalität im

Sinne ihres eigenen Domänenmodells bietet. Die Schicht kommuniziert über die bestehende Schnittstelle mit dem anderen System, wobei wenig oder keine Änderungen am anderen System erforderlich sind. Intern übersetzt die Schicht bei Bedarf in beide Richtungen zwischen den beiden Modellen. [Eva07, S. 364 - 370] [Wol16, S. 47]

Seperate Ways

Wenn der Nutzen im Vergleich zu den Integrationskosten zu gering ist, kann es eine gültige Lösung sein, zu erklären, dass ein abgegrenzter Kontext überhaupt keine Verbindung zu anderen hat. Anstatt Lösungen für Probleme im Bereich zu teilen, müssen diese in einem einfachen, spezialisierten Weg innerhalb dieses kleinen Rahmens implementiert werden. [Eva07, S. 371 - 373] [Wol16, S. 47]

Open Host Service

Wenn ein Teilsystem mit vielen anderen integriert werden muss, kann die Anpassung eines Übersetzers für jedes einzelne problematisch skalieren. In diesem Fall ist die Definition eines offenen Protokolls von Diensten, das Zugang zum Teilsystem bietet, die bevorzugte Gestaltung. [Eva07, S. 374] [Wol16, S. 48]

Published Language

Das direkte Übersetzen der Sprache von Modellen kann zu übermäßig komplexen Strukturen führen. Auch wenn eine Sprache als Datenaustauschsprache verwendet wird, wird sie festgelegt und kann nicht mehr an die Bedürfnisse der Entwicklung angepasst werden. Daher kann eine gut dokumentierte gemeinsame Sprache als allgemeines Kommunikationsmedium veröffentlicht werden. [Eva07, S. 375-378] [Wol16, S. 48]

2.2 Microservices

2.2.1 Definition

Der zur Modularisierung eingesetzte Ansatz der Microservices erfüllt verschiedene Grundlegende Vorraussetzungen: In ihrer Implementierung gibt es keine technologischen Einschränkungen. Die Dienste können unabhängig voneinander überarbeitet und auf ein Zielsystem ausgerollt werden. Weiterhin sollen Build und Deployement automatisiert sein. Sie besitzen einen eigenen Datenhaushalt sowie eigenständige Prozesse auf der ausführenden Infrastruktureinheit. Microservices nutzen verteilte Netzwerkkommunikation, um sich mit anderen Diensten zu verständigen. Transaktionen zeichnen sich durch die ACID-Eigenschaften aus, zu denen Atomizität, Konsistenz, Isolation und Dauerhaftigkeit gehören. Es ist wichtig, dass Daten konsistent bleiben. Deshalb sollten keine Transaktionen über mehrere Services hinweg durchgeführt werden. Um diese Konsistenz zu gewährleisten, können Kompensationstransaktionen eingesetzt werden. [Wol16, S. 2] [Wol16, S. 31-37]

Jeder Microservice soll eine eigene fachliche Einheit bilden, die so geschnitten ist, dass für Änderungen oder neue Features nur ein Microservice geändert werden muss. Weiterhin sollen die aus dem DDD bekannten Konzepte des Strategic Design verwendet werden. Micorservices sollen jeweils eine fachliche Domäne modellieren, sodass neue Features oder Änderungen nur in einem Microservice umgesetzt werden müssen. Ein Bounded Context kann in mehrere Microservices aufgeteilt werden, wenn das sinnvoll erscheint. [Wol16, S. 48-51]

Damit Microservices unabhängig voneinander in Produktion gebracht werden können, ist eine gekapselte Hardware nötig, welche durch Virtualisierung umgesetzt werden kann. Dies setzt eine passende Infrastruktur voraus, die automatisiert eine große Anzahl virtueller Maschinen erzeugen kann. Weiterhin benötigt jeder Microservice, als Folgerung aus der Voraussetzung, unabhängig deployen zu können, eine eigene Continuous-Delivery-Pipeline. Zudem muss es ein umfassendes Monitoring geben, da durch die Vielzahl an Anwendungen sonst Probleme nicht erkannt werden können. Dieses sollte neben technischen Metriken auch Anwendungsmetriken erfassen, um eine umfassendere Problemübersicht zu bekommen. Auch eine unabhängige Versionskontrolle der Dienste ist notwendig, da aus der unabhängigen Deploybarkeit auch verschiedene Versionierungen möglich sind. [Wol16, S. 80-81]

2.2.2 Architektur von Microservice-Systemen

Gemäß den DDD-Prinzipien kann eine fachliche Architektur erfolgen. Dabei soll es zwischen den Microservices eine lose Kopplung geben, wodurch sie isoliert und leichter abänderbar sind. Innerhalb eines Dienstes sollen die Bestandteile eng zusammenarbeiten, was einer hohen Kohäsion entspricht. Weiterhin sind zyklische fachliche Abhängigkeiten in der Architektur problematisch, da dadurch die isolierte Änderbarkeit nicht mehr gewährleistet ist. [Wol16, S. 101-105]

Verschiedene etablierte Methoden existieren im Zusammenhang mit Änderungen der Gesamtarchitektur, die durch neue Erkenntnisse aus der Domäne notwendig sein können. So kann in mehreren Diensten gemeinsam verwendeter Code in Bibliotheken ausgelagert werden. Dabei muss jedoch auf die dadurch entstehende Abhängigkeit geachtet werden, da durch unkoordinierte Änderungen Fehler in Teilen des Microservice-Systems einschleichen können. Weiterhin kann Code eines Dienstes in einen anderen übertragen werden, um Verbesserungen im Hinblick auf die Kohäsion in einem Dienst zu erreichen. Eine Alternative zur Bibliothek ist es, Code in einen gemeinsamen, neuen Dienst zu übertragen, was bei vorausgesetzter passender Schnittstellendefinition dem Ziel der entkoppelten Dienste zuträglich ist. Es ist auch möglich, alten Code in einen neuen Service zu migrieren oder die notwendige Änderung der Architektur mit einem vollständig neu entwickelten Microservice umzusetzen. [Wol16, S. 112-120]

Damit Microservices ihre gemeinsame Logik implementieren können, müssen sie sich gegenseitig aufrufen. Dabei kann eine ereignisgetriebene Architektur (Event-Driven Architecture, EDA) verwendet werden, bei der ein Dienst bei einem Ereignis aus der Anwendungslogik, einem Event, dieses versendet. Dieser Dienst, der Event-Emitter, signalisiert damit allen interessierten Microservices (Event-Consumern), dass etwas passiert ist. Dieses Vorgehen kapselt die Anwendungslogik und entkoppelt die Bearbeitung der Events zeitlich, was dem Ziel einer losen Kopplung beiträgt. [Wol16, S. 137 - 138]

Die Konfiguration eines Microservice-Systems wird mit steigender Anzahl von Services entsprechend aufwendig. Mit Werkzeugen wie Zookeeper, etcd oder Spring Cloud Config werden Key-Value-Stores bereitgestellt, die dies vereinfachen. Jedoch gilt es zu beachten, dass dadurch eine Abhängigkeit zu einem Konfigurationsserver entsteht. Alternativ kann eine Konfiguration auf dem Server des Microservices selbst vorliegen. Dies vermeidet die Notwendigkeit eines zentralen Konfigurationsservers, kann aber auch Herausforderungen in Bezug auf die Synchronisation und Verwaltung der Konfigurationen über mehrere Instanzen hinweg mit sich bringen. [Wol16, S. 141 - 142]

Service Discovery stellt sicher, dass Microservices sich gegenseitig finden können. Dazu muss diese dynamisch sein um gehende und kommende Microservices zu finden. Technolgien für diese sind besipielsweiße DNS (Domain Name Sstem), Eureka, Consul. [Wol16, S. 143 - 146]

Bei hoher Last können Microservices unabhängig skalieren. Dadurch müssen mehrere Instanzen registriert werden, die sich die Last gegen einen Dienst teilen. Um HTTP anfragen in solch

einem Fall zu verteilen wird ein Load Balanecr genutzt, der sich nach außen wie eine einzige Instanz verhält, aber die Anfragen auf mehrere verteilt. Da bei einem einzigen Load Balancer für mehrere Microservices, dieser zum Flaschenhals werden kann, sollte pro Microservice ein Load Balancer existieren. Technologien für Load Balancer existieren nativ in Cloud Umgebungen oder können beispielsweiße mit Apache httpd, nginx oder HAProxy umgesetzt werden. [Wol16, S. 147 - 150]

Die Sicherheitsarchitektur muss einheitlich auf der Gesamtebene des Systems definiert werden, um sicherzustellen, dass ein Zugriff eines Nutzers einheitlich behandelt wird. Dazu gehört es, Authentifizierung und Autorisierung zu gewährleisten. Der einzelne Microservice des fachlich abgebildeten Systems soll keine Authentifizierung übernehmen. Stattdessen wird diese Aufgabe von einem zentralen Server wahrgenommen. Bei der Autorisierung ist ein Zusammenspiel gefragt, da über Benutzergruppen bestimmte Features des Microservices zugewiesen sind. Welche dies sind, soll der Microservice entscheiden. Eine Implementierung dieses Konzepts ist das OAuth2-Protokoll. Dabei erhält ein Nutzer nach erfolgreicher Authentifizierung und Autorisierung ein Access Token, welches genutzt wird, um mit einem Service zu interagieren. Dieses kann weiterhin von einem Microservice an andere Microservices weitergereicht werden, um berechtigte Anfragen zu ermöglichen und die einheitliche Sicherheitsrichtlinie im gesamten System zu gewährleisten. [Wol16, S. 153 - 161]

2.2.3 Kommunikation

Mit REST (representational State Transfer) können Microservices untereinander Code aufrufen. Es gibt eine Vielzahl von Ressourcen, die über URIs identifiziert werden können. Die Ressourcen können über eine feste Menge von methoden manipuliert werden, welche z.B. durch HTTP vorgegeben werden. Ressourcen können unterschiedlich repräsentiert werden. Beziehungen zwischen Ressourcen können durch Links angegeben werden(HATEOAS). Der Server in einem REST System sollte Zustandslos sein. Eine RESTful-HTTP Schnittstelle kann durch einen Cache ergängzt werden. RESTful HTTP ist synchron. Daher können Ausfälle auftreten, welchen mit Timeouts entgegengewirkt werden kann um Fehlerfortpflanzung zu vermeiden. [Wol16, S. 179-182]

Mit REST (Representational State Transfer) können Microservices untereinander Code aufrufen. Es gibt eine Vielzahl von Ressourcen, die über URIs identifiziert werden können. Die Ressourcen können über eine feste Menge von Methoden manipuliert werden, welche z. B. durch HTTP vorgegeben sind. Ressourcen können unterschiedlich repräsentiert werden. Beziehungen zwischen Ressourcen können durch Links angegeben werden (HATEOAS). Der Server in einem REST-System sollte zustandslos sein. Eine RESTful-HTTP-Schnittstelle kann durch einen Cache ergänzt werden. RESTful HTTP ist synchron. Daher können Ausfälle auftreten, denen mit Timeouts entgegengewirkt werden kann, um Fehlerfortpflanzung zu vermeiden. [Wol16, S. 182 - 183]

Messaging-Systeme sind eine weitere Möglichkeit, um Kommunikation zwischen Microservices umzusetzen. Diese basieren auf dem Verschicken von Nachrichten. Diese können auch beim Ausfall des Netzwerks übertragen werden, da Messaging-Systeme diese zwischenspeichern. Auch eine korrekte Verarbeitung kann garantiert werden, indem eine erneute Übertragung im Fehlerfall ausgelöst wird. Solche Messaging-Architekturen sind asynchron und gut auf hohe Latenzzeiten abgestimmt. Der Aufruf eines anderen Service blockiert die weitere Verarbeitung nicht. Daher kann ein Microservice weiterarbeiten, ohne auf eine Antwort warten zu müssen. Der Sender kennt den Empfänger nicht, da die Nachricht auf einer Queue oder einem Topic landet. Dort registrieren sich mögliche Empfänger. Dadurch sind Sender und Empfänger entkoppelt. Messaging bietet eine Lösung für transaktionale Systeme mit Microservices. Das Schicken und Empfangen von Nachrichten ist dabei Teil einer Transaktion. Wenn beim Verarbeiten der Nachricht ein Fehler auftritt, werden alle ausgehenden Nachrichten storniert und Änderungen rückgängig gemacht. Auch die Empfänger können transaktional abgesichert sein. Entscheidend ist, dass Schicken und Empfangen von Nachrichten und die Transaktion auf der Datenbank in einer Transaktion zusammengefasst werden. Messaging kann durch verschiedene Technologien wie AMQP, Apache Kafka, JMS oder ATOM Feeds umgesetzt werden. [Wol16, S. 183 - 187]

Jeder Microservice kann eine oder mehrere Schnittstellen für andere Microservices bereitstellen. Außerdem kann ein Microservice Schnittstellen nach außen haben, die Interaktionen außerhalb des Systemkontexts ermöglichen. Wenn ein Microservice eine Schnittstelle ändert, sollte er auch noch die alte Schnittstelle unterstützen. Weiterhin sollten Schnittstellen getrennt umgesetzt

werden. Um Änderungen an einer Schnittstelle deutlich zu machen, kann eine Versionsnummer genutzt werden. Semantic Versioning definiert dabei eine Semantik für Versionsnummern, wobei das Format MAJOR.MINOR.PATCH lautet. Dabei sind Änderungen der Major-Version nicht rückwärtskompatibel, Änderungen der Minor-Version bieten neue Features, und Patch-Änderungen beheben Fehler (Bugfixes). Die Version sollte nicht in einer URL, sondern besser in einem Header bei einem REST-Request angegeben werden. [Wol16, S. 190 - 192]

2.2.4 Architektur eines Microservices

- Fachliche Architektur [Wol16, S. 197 - 198]

- CQRS [Wol16, S. 199 - 201]

- Event Sourcing [Wol16, S. 201 - 203]

- Hexagonale Architektur [Wol16, S. 203 - 206]

- Resilience und Stabilität [Wol16, S. 207 - 212]

2.2.5 Betrieb und Continious Delivery

- Logging [Wol16, S. 244 - 249]

- Monitoring [Wol16, S. 250 - 256]

- Deployement [Wol16, S. 256 - 260]

- Infrastruktur [Wol16, S. 263 - 269]

2.3 Cloud-Infrastruktur

2.3.1 Definition

- Cloud, Virtualisierung, Continous Delivery, Container

[Ste20, S. 20] [Ste20, S. 46-63]

- Docker [Ste20, S. 131-166]

- Kubernetes [Ste20, S. 166-192]

2.4 Modellgetriebene Entwicklung von Microservices

2.4.1 AjiL - an Eclipse tool for MSA modeling with a code generator

AjiL is an Eclipse-based tool for graphical MSA modeling. It features a diagram editor for modeling rudimentary domain models, functional and infrastructure microservices, and provided and required interfaces. Thus, AjiL facilitates creating intermediate models for service interfaces (see the section “Intermediate Models”). AjiL ships with a code generator that produces microservice implementations from AjiL models. The generated Java code is based on Spring Boot and Spring Cloud. It is directly runnable and comprises interface implementations, stubs for implementing services’ business logic, and configuration files for infrastructure components. However, AjiL currently does not support round-trip engineering.4 The tool is available at github.com/SeelabFhdo/AjiL.

https://ieeexplore.ieee.org/abstract/document/8354426 https://github.com/SeelabFhdo/AjiL

2.4.2 OCCIware - a unified framework for managing diverse cloud resources and services

Cloud computing has emerged as the main paradigm for hosting and delivering computing resources as services over Internet. It provides a delivery model for computing resources at infrastructure, platform, and software levels. However, there is a plethora of cloud providers offering different resource management interfaces. Due to this diversity, the consumption, provisioning, management, and supervision of cloud resources are subjected to four key issues, i.e., heterogeneity, interoperability, integration, and portability. To cope with these issues, Open Cloud Computing Interface (OCCI) is proposed as a community-based and open recommendation standard for managing any kind of cloud resources. Currently, only runtime implementations exist for OCCI, and each one targets a specific cloud service model such as Infrastructure as a Service (IaaS), Platform as a Service (PaaS), or Software as a Service (SaaS). Thus, OCCI lacks an approach to model and execute different OCCI artifacts. Our approach provides a generic modeling framework coupled with a generic runtime implementation. In this article, we propose the OCCIware approach, which represents the first approach to design, validate, generate, implement, deploy, execute, and supervise everything as a service with OCCI. This approach provides OCCIware Studio, the first model-driven tool chain for OCCI. It is built around OCCIware Metamodel, which defines the static semantics for the OCCI standard in Ecore and OCL. In addition, it proposes OCCIware Runtime, the first generic OCCI runtime implementation targeting all the cloud service models (IaaS, PaaS, and SaaS). OCCIware provides a unique and unified framework to manage OCCI artifacts and, at the same time, it represents a factory to build cloud domain-specific modeling frameworks where each framework targets a specific cloud domain. OCCIware has been applied in various cloud domains and use-cases, which validate its applicability.

https://www.sciencedirect.com/science/article/pii/S0167739X18306071# sec5

2.4.3 ARGON - automated cloud infrastructure provisioning using a domain-specific language

Infrastructure as Code (IaC) is an approach to infrastructure automation based on practices from software development. IaC tools use scripts to specify the creation, update, and execution of cloud infrastructures. As each cloud provider offers a different type of infrastructure, the definition of an infrastructure element implies to write several lines of code that strongly depend on the target cloud provider. As a result, managing IaC scripts has become a time-consuming and error-prone activity. In previous work, we have presented ARGON, which is a model-driven infrastructure provisioning tool. ARGON implements a domain-specific language (DSL) for modeling the characteristics of the cloud infrastructure and provides transformation engines to automate the infrastructure provisioning for different cloud providers. In this demonstration, we show the use of ARGON for modeling and provisioning a load balancer in Amazon Web Services and Microsoft Azure. The load balancer distributes incoming application traffic across multiple virtual machines.

https://ieeexplore.ieee.org/abstract/document/8904494

2.4.4 model-driven approach for adaptive microservice-based systems using VxBPMN4MS

A microservice-based system is composed of numerous independently deployed and executed microservices, among which normally exist the complex dependencies. Traditional service composition approaches usually expect the business process predefined at design time. As a result, it is difficult for the microservice-based system to quickly adapt to the frequently changing operation environments and business requirements. To address the above limitations, we propose a variability-enabling and model-driven approach to developing adaptive microservice-based systems. Our approach first models the business process with variability using VxBPMN4MS, an extension of Business Process Model and Notation (BPMN) with support for variability, then transforms the business process model to variability supported microservice composition frameworks, and finally derives business process instances at run-time according to the predefined process configuration. We have developed a platform to automate the proposed approach as much as possible, and conducted a case study to evaluate the effectiveness of the proposed approach and platform.

https://ieeexplore.ieee.org/abstract/document/9529482

2.4.5 Context Mapper, a tool for DDD and microservice design.

Service-oriented architectures and microservices have gained much attention in recent years; companies adopt these concepts and supporting technologies in order to increase agility, scalability, and maintainability of their systems. Decomposing an application into multiple independently deployable, appropriately sized services and then integrating such services is challenging. With strategic patterns such as Bounded Context and Context Map, Domain-driven Design (DDD) can support business analysts, (enterprise) architects, and microservice adopters. However, existing architecture description languages do not support the strategic DDD patterns sufficiently; modeling tools for DDD primarily focus on its tactical patterns. As a consequence, different opinions on how to apply strategic DDD exist, and it is not clear how to combine its patterns. Aiming for a clear and concise interpretation of the patterns and their combinations, this paper distills a meta-model of selected strategic DDD patterns from the literature. It then introduces Context Mapper, an open source project that a) defines a Domain-specific Language (DSL) expressing the strategic DDD patterns and b) provides editing, validation, and transformation tools for this DSL. As a machine-readable description of DDD, the DSL provides a modeling foundation for (micro-)service design and integration. The models can be refactored and transformed within an envisioned tool chain supporting the continuous specification and evolution of Context Maps. Our validation activities (prototyping, action research, and case studies) suggest that the DDD pattern clarification in our meta-model and the Context Mapper tool indeed can benefit the target audience.

https://www.scitepress.org/Papers/2020/89105/89105.pdf

2.4.6 DOML simplifies infrastructure deployments by unifying multiple IaC languages

One of the main DevOps practices is the automation of resource provisioning and deployment of complex software. This automation is enabled by the explicit definition of Infrastructure-as-Code (IaC), i.e., a set of scripts, often written in different modelling languages, which defines the infrastructure and applications to be deployed.

We introduce the DevOps Modelling Language (DOML), a new Cloud modelling language for infrastructure deployments. DOML is a modelling approach that can be mapped into multiple IaC languages, addressing infrastructure provisioning, application deployment and configuration at once. The idea behind DOML is to use a single modelling paradigm which can help to reduce the need of deep technical expertise in using different specialised IaC languages.

We present the DOML's principles and discuss the related work on IaC languages. We demonstrate the DOML advantages for the end-user in comparison with state-of-the-art IaC languages such as Ansible, Terraform, and Cloudify, and show its effectiveness through an example.

Literaturverzeichnis

[Eva07] Eric Evans. *Domain-driven design: tackling complexity in the heart of software*. Addison-Wesley, Upper Saddle River, NJ [u.a.], 11. printing edition, 2007. Eric Evans.

[Ste20] Daniel Stender. *Cloud-Infrastrukturen: das Handbuch für DevOps-Teams und Administratoren*. Rheinwerk computing. Rheinwerk Computing, Bonn, 1. auflage edition, 2020. Daniel Stender.

[Wol16] Eberhard Wolff. *Microservices: Grundlagen flexibler Softwarearchitekturen*. dpunkt.verlag, Heidelberg, 1., korrigierter nachdruck edition, 2016. Eberhard Wolff.